

Macro-to-micro behavioural mappings in distributed systems

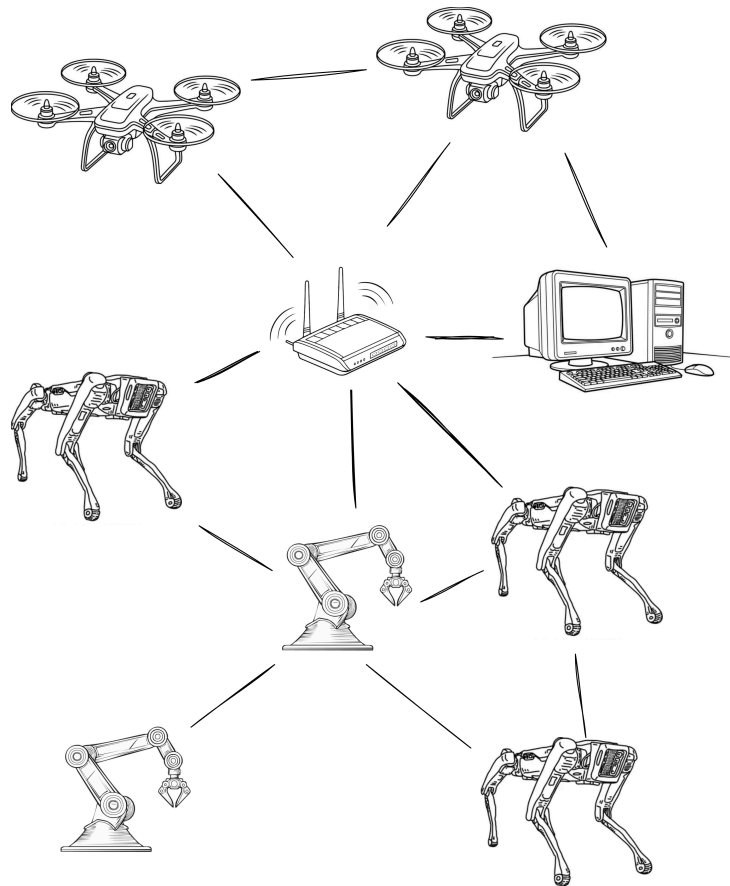
A characterisation on event structures

Context

- Collective systems
- Programming paradigms

Goal

- Common, unified framework
- Simpler share of ideas



Examples

Aggregate computing

```
// Gradient field computation
def distanceTo(s) {
  rep( $\infty$ ) {
    (dist)  $\Rightarrow$  mux(s, 0, minHood(nbr{dist} + nbrRange()))
  }
}
```

- Program into common instructions
- Abstract concept of field

Choreography

```
// Conditional information sharing
C = if p.c = x
  then (p  $\rightarrow$  r[L]; p.c  $\rightarrow$  r; 0)
  else (p  $\rightarrow$  r[R]; r.c  $\rightarrow$  p; 0)
```

- Program into device-specific instructions
- Explicit concept of message / interaction

Two formalisms to express the behavior of collective systems that are not mutually intelligible.

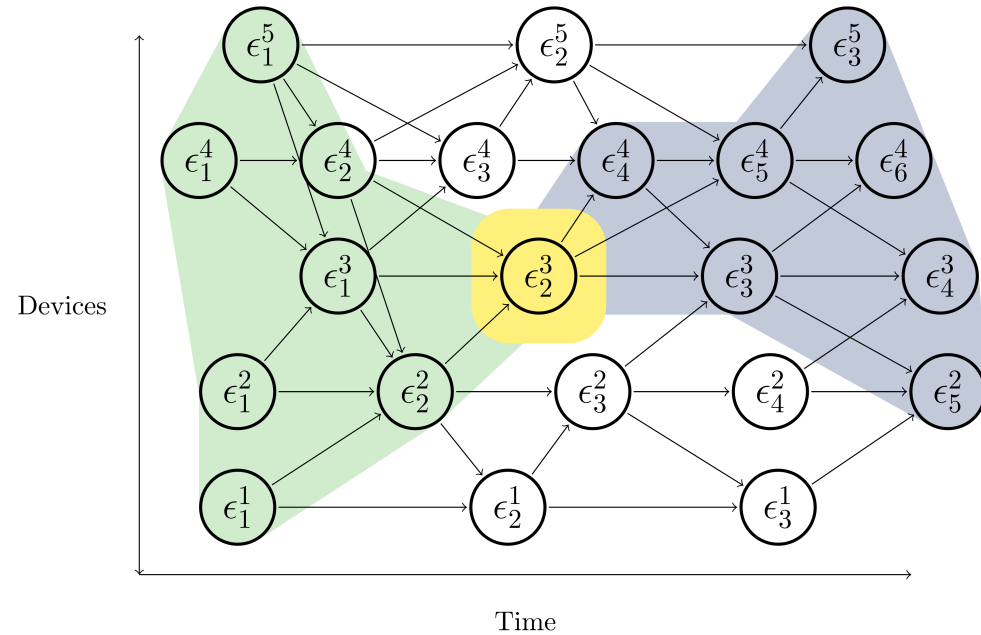
A unifying framework

To capture and describe:

- Expected scenarios – macro-input
- Desired outcomes – macro-output and effect
- Deployment process – macro-program to device policy
- Adaptation process – policy evolution over time

Event structure

- Captures the execution of a collective system
- Express execution in function of events (*sense*, *compute*, *(inter)act*)



Expected scenarios – *macro-input*

The *initial* conditions that affect the system dynamics:

- environmental state
- system state
- network topology

Desired outcomes – *macro-output* and *effect*

- Macro-output: informative result of the collective computation

function of final events: $f(\epsilon_{T_{end}}^*)$

- Macro-effect: changes on the environment caused by the system execution

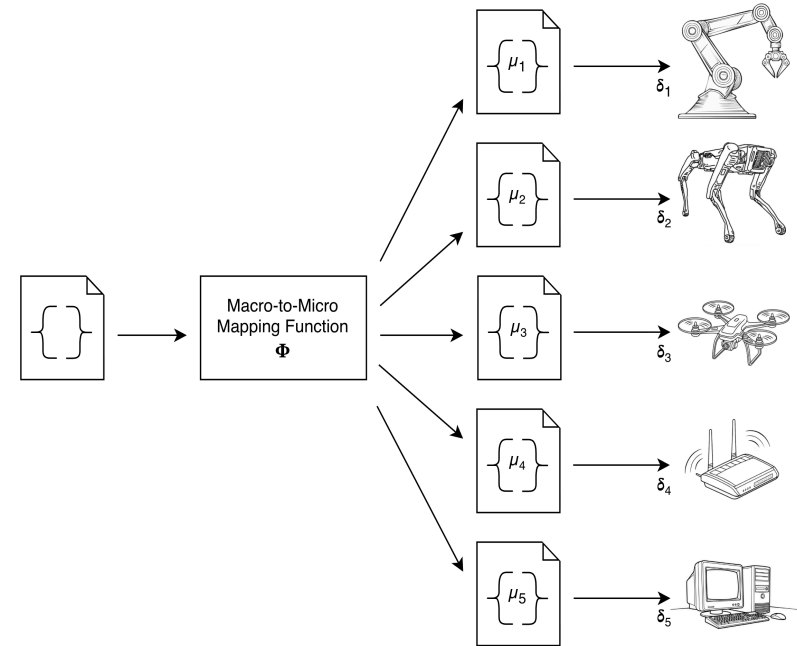
function of the final collective perception: $f(s_{T_{end}}^*)$

Macro-programming goal

- Compute the *macro-output* or cause the *macro-effect* starting from *macro-inputs*
- Map space-time circumstances to actions (i.e., define specific behavioral policies)

Localization process & compatibility condition

- Map devices to policies – decide which device will execute a specific policy
- Deploy policies on devices capable of executing them:
 $requirement(policy) \subseteq capability(device)$



Characterization of macro-programs

According to time:

- static: the policy of each device remains the same over time
- dynamic: the policy of at least one device changes during execution

According to the collective:

- homogeneous: all devices share, at any moment, the same policy
- heterogeneous: at least two devices employ, at any moment in time, different policies

Results

Aggregate computing

```
// Gradient field computation
def distanceTo(s) {
  rep( $\infty$ ) {
    (dist)  $\Rightarrow$  mux(s, 0, minHood(nbr{dist} + nbrRange()))
  }
}
```

- *Homogeneous* localization
- Common device capability and unique requirement sets
- Events capture field-based computation

Choreography

```
// Conditional information sharing
C = if p.c = x
  then (p  $\rightarrow$  r[L]; p.c  $\rightarrow$  r; 0)
  else (p  $\rightarrow$  r[R]; r.c  $\rightarrow$  p; 0)
```

- *Heterogeneous* localization
- Requirement and capability sets associated with device type / role
- Events capture message-based computation

Results

- Unified interpretation of different paradigms
- Possibility to propose approach-agnostic solutions
- Simplified sharing of ideas

Future works

- Further elaborate the macro-to-micro mapping mechanism
- Further elaborate dynamic macro-programming and enabling techniques

Acknowledgments

This work has been founded by the project "FoMaSE: Foundations for Macro-programming-based Software Engineering".